The Building Blocks of LLMs

A Mathematical History

From Archimedes to Attention:
2,000+ Years of Mathematics Inside Your AI

Prof. J. Osterrieder 2026

What's Inside Your AI?

When you type a question to ChatGPT or Claude...

The machine that answers is built entirely from mathematics.

- Not magic
- Not "intelligence"
- **Specific math**, invented by **specific people**, at **specific times**

Smartphone screen showing an AI chat. Floating labeled arrows point to different parts of the response: "Linear Algebra," "Calculus," "Probability," "Information Theory," "Logic," "Function Approximation"

Key Insight

You already know some of this math

The Exploded Transformer — What We Will Build Today

EXPLODED TRANSFORMER DIAGRAM (full-slide).
Color-coded components: Matrix multiplication layers (blue, Linear Algebra) — Dot product scores (blue) — Gradient descent arrows (green, Calculus) — Softmax probability bars (orange, Probability) — Cross-entropy loss (red, Information Theory) — Boolean gates / GPU chips (gray, Logic) — Activation function curves (purple, Function Approx.) — Neuron nodes (yellow, The Neuron) — Residual connections / skip arrows (yellow) — Layer normalization (yellow) — Attention arrows (teal, Sequence Modeling).
Intentionally overwhelming on first view.

This looks complicated now. By the end, you will know every piece — and WHO invented it.

Our Roadmap — The Convergence Diagram

CONVERGENCE DIAGRAM – INITIAL STATE. Central node: large "?" (gray). 10 gray nodes in a semicircle around it, each labeled: (1) Linear Algebra [1809–1900s], (2) Calculus & Optimization [c. 250 BCE–1986], (3) Probability & Statistics [1654–1933], (4) Information Theory [1865–1948], (5) Logic & Computation [1854–1970s], (6) Function Approximation [1807–1991], (7) The Neuron & Depth [1943–2016], (8) Sequence Modeling [1906–2014], (9) The Transformer [2017], (10) Scaling to LLMs [2018–2026]. Gray arrows from each node toward center "?" node. All nodes are dim/gray — none lit up yet.

Linear Algebra

The Skeleton of Neural Networks

1809–1900s

"Neural networks are, at their core, matrix multiplication machines."

What Is a Vector? — Forces, Velocities, and Words

A **vector** is a list of numbers that represents something.

Hermann Grassmann (1809–1877)

Origin: Stettin, Prussia (now Szczecin, Poland)
Key work: *Die lineale Ausdehnungslehre* (1844) — the first general theory of vector spaces in any number of dimensions.
A schoolteacher whose work was so ahead of its time that almost nobody read it.

Left: a 2D vector arrow. Middle: a 3D vector arrow. Right: "768-dimensional vector" with fading dots. Below: "In an LLM, each word becomes a vector — a list of 768+ numbers."

Sir William Rowan Hamilton (1805–1865)

Origin: Dublin, Ireland

Plaque on Brougham Bridge showing Hamilton's quaternion

Modern Vector Notation — Gibbs and Heaviside

Josiah Willard Gibbs (1839–1903)

Origin: New Haven, Connecticut, USA
Key work: Extracted the useful vector parts from Hamilton's quaternions; created modern vector analysis (lecture notes *Elements of Vector Analysis*, c. 1881–1884).

Oliver Heaviside (1850–1925)

Origin: London, England
Key work: Self-taught electrical engineer who independently developed vector calculus; reformulated Maxwell's equations into

Side-by-side comparison: LEFT — Hamilton's quaternion notation (complex, multi-line). RIGHT — Gibbs's vector notation for the same operation (clean, compact). The simplification is visually obvious. Caption: "Same math. Simpler notation."

Matrices — Cayley and Sylvester

Arthur Cayley (1821–1895)

Origin: Richmond, Surrey, England
Key work: "A Memoir on the Theory of Matrices" (1858) — established matrices as algebraic objects in their own right.
Also a practicing lawyer for 14 years before returning to academia.

James Joseph Sylvester (1814–1897)

Origin: London, England
Key work: Coined the word "matrix" (1850),

TOP: A simple 3×3 matrix, annotated: "This is a matrix. Cayley (1858) made it a mathematical object." BOTTOM: A schematic LLM weight matrix (much larger, dots indicating millions of entries): "This is also a matrix. Millions of entries. Same math."

Matrix Multiplication — The Core Operation

Carl Friedrich Gauss (1777–1855)

Origin: Brunswick, Germany
Key work: Gaussian elimination (c. 1809–1810) for solving linear systems — laid groundwork for computational linear algebra. Originally used for astronomical orbit calculations.

Key Insight

Every neural network layer is a matrix multiplication followed by a nonlinear function. That's it.

TOP: Step-by-step diagram of matrix multiplication — a 3×3 matrix times a 3×1 vector, showing dot products row by row. BOTTOM: The SAME operation drawn as a neural network layer — inputs on the left, weights as connections, outputs on the right. Caption: "Same math. Different picture."

The Dot Product — Measuring Similarity

Recipe (the dot product in plain language):

1. Take two lists of numbers, same length
2. Multiply matching entries: first × first, second × second, ...
3. Add up all the products
4. Result is **one number**:

Big positive	= very similar
Near zero	= unrelated
Big negative	= opposites

Formal: $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$

Two 2D arrows in three configurations: (1) pointing same direction → large positive dot product, (2) perpendicular → zero, (3) opposite → large negative. Caption: "The dot product measures how much two vectors agree."

LLM context: "The Query vector for 'it' and the Key vector for 'cat' have a large dot product. The model attends 'it' to 'cat.' This is how it

Cosine Similarity — Dot Product, Normalized

Recipe:

1. Compute the dot product
2. Divide by the length of each vector
3. Result is always between -1 and $+1$

Think of it as: *how much do these two vectors point the same way?* (Ignoring how long they are.)

Formal: $\cos(\theta) = \dfrac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}|\,|\mathbf{b}|}$

Key Insight

A 2D embedding space with word vectors as points. "King" and "queen" are close (high cosine similarity). "King" and "banana" are far apart (low cosine similarity). Angle arcs drawn between vector pairs. Caption: "Cosine similarity = the angle between word vectors."

Eigenvalues — The Natural Modes [EXTENDED]

Leonhard Euler (1707–1783)

Origin: Basel, Switzerland / St. Petersburg / Berlin
Key work: First encountered eigenvalue-like concepts studying vibrating strings and rotating bodies (1740s–1750s).

Augustin-Louis Cauchy (1789–1857)

Origin: Paris, France
Key work: Proved symmetric matrices have real eigenvalues (1829).

A 2D linear transformation shown as arrows: most arrows change direction, but the EIGENVECTOR arrows only stretch or shrink. Caption: "Eigenvectors are the directions a transformation doesn't rotate."

LLM connection: Principal Component Analysis (PCA) uses eigenvalues to visualize and understand what LLMs learn. "What does this neuron represent?" — eigenvalue analysis helps answer.

High-Dimensional Spaces — Where Words Live

LLMs represent words as vectors in spaces with **hundreds or thousands of dimensions**.

The math works the same as in 2D or 3D — we just can't draw it.

Richard Bellman (1920–1984)

Origin: New York City, USA
Key work: Coined "the curse of dimensionality" (c. 1957–1961): in high dimensions, data becomes sparse and distances behave counterintuitively.

2D scatter plot of word embeddings: "king," "queen," "man," "woman" as labeled points. Arrow showing the famous analogy: king − man + woman = queen. Caption: "Projected from 768 dimensions down to 2. In the full space, THOUSANDS of such relationships exist simultaneously."

Tomas Mikolov et al. demonstrated this with **Word2Vec** (2013) — word vectors trained on large text corpora capture semantic relationships as *linear directions*.

Quick Experiment — Word Vector Arithmetic

What does this equal?

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} = ?$$

Space for student responses. After reveal: show the embedding space diagram with the parallelogram connecting king, man, woman, queen. Large text: "QUEEN" with a checkmark. Attribution: "Mikolov et al., Word2Vec, 2013"

From Words to Numbers — The Embedding

Step-by-step pipeline diagram (full-width): Step 1: User types: "The cat sat on the mat" Step 2: Tokenizer splits it: ["The", "cat", "sat", "on", "the", "mat"] Step 3: Each token becomes a vector: [0.23, −0.87, 0.44, . . .] (768 numbers) Step 4: The sentence becomes a MATRIX: 6 rows (tokens) × 768 columns (dimensions) Final caption: "Your sentence is now a matrix. Everything that follows is linear algebra."

Historical lineage: Grassmann (1844, abstract vector spaces) → Mikolov (2013, Word2Vec learned embeddings) → Modern LLM embedding layers (2017–present, learned jointly with the transformer)

Attention as Matrix Multiplication — A Preview

The attention recipe (conceptual pipeline):

1. Start with input matrix X (sentence as matrix)
2. Multiply by $W_Q \to$ get Queries
   (matrix multiplication — Cayley)
3. Multiply by $W_K \to$ get Keys
   (matrix multiplication — Cayley)
4. Multiply by $W_V \to$ get Values
   (matrix multiplication — Cayley)
5. Scores = dot product of each Query with each Key
   (dot product — Gibbs, Slide 10)
6. Normalize scores with softmax

Simplified attention diagram showing the pipeline: $X \to W_Q \to Q$, $X \to W_K \to K$, $X \to W_V \to V$, then $Q \cdot K^T \to$ scores $\to$ softmax $\to \times V \to$ output. Each step labeled with its mathematical origin and section reference.

Building the Diagram — Linear Algebra Complete

CONVERGENCE DIAGRAM UPDATE: The "Linear Algebra" node now lights up in BLUE. Label appears: "Matrix multiplication = neural network layers." All other 9 nodes remain gray. Central node still "?"

Summary — who built the skeleton:

1843	Hamilton — quaternions
1844	Grassmann — vector spaces
1850	Sylvester — named "matrix"
1858	Cayley — matrix theory
1880s	Gibbs/Heaviside — notation
1809+	Gauss — elimination
2013	Mikolov — Word2Vec

Key Insight

Every layer of an LLM is a matrix multiplication. Every attention score is a dot product. The

Calculus & Optimization

How Neural Networks Learn

c. 250 BCE – 1986

"A neural network learns by rolling downhill on a landscape made of calculus."

Roots in Antiquity — Archimedes and Infinitesimals

Archimedes of Syracuse

Origin: Syracuse, Sicily
Key work: The "method of exhaustion" — calculated areas and volumes by approximating with ever-finer polygons.

His lost *Method* manuscript, rediscovered in 1906 in the Archimedes Palimpsest, shows he used infinitesimal reasoning more explicitly than previously known.

Archimedes' method of exhaustion: a circle with inscribed polygons (4 sides, 8 sides, 16 sides, 32 sides), visually converging to the circle's area. Caption: "Approximate, refine, repeat. This is what calculus does."

Key Insight

The core idea of calculus: approxi

The Derivative — Measuring Change

Pierre de Fermat (1601–1665)

Origin: Beaumont-de-Lomagne, France
Key work: Method for tangent lines (c. 1636–1638); found maxima and minima by setting a difference quotient to zero.

Isaac Newton (1642–1726/27)

Origin: Woolsthorpe, England
Key work: "Method of fluxions" (c. 1665–1666) — rates of change for physics.

TOP: Three notations side by side — Fermat's tangent method (geometric), Newton's fluxion dot notation, Leibniz's dy/dx. Three different notations for the same idea.

A simple curve with a tangent line. Label: "The derivative = the slope of this line = how fast things are changing."

The Chain Rule — The Key to Backpropagation

Recipe (the chain rule in plain language):

If y depends on u and u depends on x...

To find how much x affected y through the chain, multiply the effects at each step.

Formal: $$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Leibniz's notation makes this look almost obvious — the du's "cancel." (They don't really, but the intuition helps.)

Pipeline diagram: TOP: $x \to$ [function f] $\to u \to$ [function g] $\to y$ with arrows. Below: "If the error changes y, how much did x contribute? Multiply along the chain." BOTTOM: Same diagram relabeled as a neural network: "Input $\to$ Layer 1 $\to$ Hidden $\to$ Layer 2 $\to$ Output." Red backward arrows labeled "chain rule" at each step.

Key Insight

Backpropagation uses the chain rule to

Gradient Descent — Rolling Downhill [EXTENDED]

Augustin-Louis Cauchy (1789–1857)

Origin: Paris, France
Key work: Published the first description of the method of steepest descent (1847), *Méthode générale pour la résolution des systèmes d'équations simultanées.*

A 3D "loss surface" landscape with contour lines. A ball sits at a high point. Arrows show the gradient direction (steepest uphill). The ball moves in the OPPOSITE direction (steepest downhill). Multiple steps shown as a zigzagging path toward a valley. Caption: "Gradient descent: always walk downhill. Cauchy, 1847."

Recipe:

1. Compute the gradient ("which way is uphill?")
2. Take a small step in the opposite

Backpropagation — The Algorithm That Changed Everything

Seppo Linnainmaa (b. 1945)

Origin: Finland
Key work: Published automatic differentiation in his Master's thesis (1970, University of Helsinki) — the mathematical core of backpropagation.

Rumelhart/Hinton/Williams (1986)

David Rumelhart (1942–2011, USA)
Geoffrey Hinton (b. 1947, London, UK/Canada)
Ronald Williams (USA)

Multi-layer network diagram. Forward pass: blue arrows left to right. Backward pass: red arrows right to left, labeled "chain rule" at each step.

Timeline:

1847	Cauchy — steepest descent
1970	Linnainmaa — auto. differentiation
1986	Rumelhart/Hinton/Williams — backprop
2024	Hinton — Nobel Prize in Physics

Stochastic Gradient Descent — Learning from Samples

Robbins & Monro (1951)

Herbert Robbins (1915–2001, New Castle, PA, USA)
Sutton Monro (1920–1995, USA)

Published the Robbins–Monro algorithm (1951) — the first stochastic approximation method. The theoretical foundation for SGD.

Comparison diagram, side by side: LEFT: "Full gradient descent" — smooth downhill path. Label: "Must look at ALL data for each step. Too slow." RIGHT: "Stochastic gradient descent" — noisy, zigzagging path that still reaches the bottom. Label: "Look at a random sample each step. Much faster."

Key Insight

Every LLM you have ever used was trained with some variant of

The Loss Landscape — A Visual Tour

FULL-SLIDE VISUAL: A 3D rendering of a neural network loss landscape (in the style of Li et al., 2018, "Visualizing the Loss Landscape of Neural Nets"). Labeled features: — "Global minimum" (the best the network can do) — "Local minima" (good but not the best) — "Saddle points" (flat regions that look like minima but aren't) — "The path of SGD" (a noisy trajectory from a random start to a good minimum) Source: reconstructed from Li et al. 2018 or similar public-domain visualization.

This is what training an LLM looks like mathematically. The landscape has *billions* of dimensions — we can only visualize a 2D slice.

Building the Diagram — Calculus Complete

CONVERGENCE DIAGRAM UPDATE: "Linear Algebra" node lit in blue (from last section). "Calculus & Optimization" node now lights up in GREEN. Label: "Gradient descent = how it learns." Remaining 8 nodes still gray. Central node still "?"

Summary — who taught it to learn:

c. 250 BCE	Archimedes — proto-calculus
c. 1636	Fermat — tangents, extrema
1665–66	Newton — fluxions
1684–86	Leibniz — dy/dx notation
1847	Cauchy — steepest descent
1951	Robbins–Monro — SGD
1970	Linnainmaa — auto. diff.
1986	Rumelhart/Hinton/Williams — backpropagation

Key Insight

Gradient descent + backpropaga-

Session 1 Complete

We know the skeleton and how it learns. Next: how does it handle uncertainty?

Probability & Statistics
The Language of Uncertainty

1654–1933

Pascal · Fermat · Bernoulli · Bayes · Kolmogorov · Fisher · Shannon · Boltzmann

The Birth of Probability — A Gambling Problem (1654)

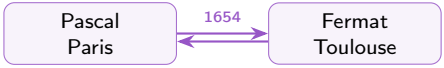
Blaise Pascal (1623–1662)

Origin: Clermont-Ferrand, France

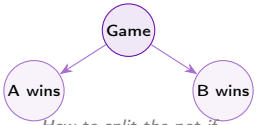
Mathematician, physicist, philosopher. Co-founded probability theory in correspondence with Fermat (1654) about the problem of points: how to divide stakes in an interrupted game of chance.

Pierre de Fermat (1601–1665)

Origin: Beaumont-de-Lomagne, France



Problem of Points



How to split the pot if the game stops early?

From Gambling to Laws — Bernoulli and Laplace

Jacob Bernoulli (1655–1705)

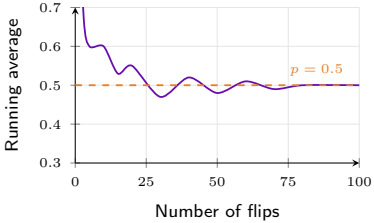
Origin: Basel, Switzerland

Proved the law of large numbers (published posthumously in *Ars Conjectandi*, 1713): as you repeat an experiment, the observed frequency converges to the true probability.

Pierre-Simon Laplace (1749–1827)

Origin: Beaumont-en-Auge, France

Théorie analytique des probabilités



Key Insight

Why do LLMs improve with more data? Bernoulli's law: more data ⇒ learned probabilities converge to the true patterns of language.

Bayes' Theorem — Updating Beliefs with Evidence (1763)

Thomas Bayes (c. 1701–1761)

Origin: London, England

Presbyterian minister and mathematician. *Essay towards solving a Problem in the Doctrine of Chances* published **posthumously** (1763) by his friend Richard Price.

Formal

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Surprise Example

Test accuracy: 99%
Disease prevalence: 1 in 10,000
Positive test ⇒ chance you have it?

Answer: ∼1%
Most positives are *false* positives!

Recipe: Updating Beliefs

New belief =

$$\frac{\text{(how likely the evidence if my guess is right)} \times \text{(original guess)}}{\text{(how likely the evidence overall)}}$$

Key Insight

Kolmogorov's Axioms — Probability Made Rigorous (1933)

Andrey Kolmogorov (1903–1987)

Origin: Tambov, Russia

Published *Grundbegriffe der Wahrscheinlichkeitsrechnung* (Foundations of the Theory of Probability, 1933), axiomatizing probability using measure theory.

This put probability on the same rigorous footing as geometry (Euclid) and calculus (Cauchy/Weierstrass).

Kolmogorov's Three Axioms

1. **Non-negativity:** Probabilities are between 0 and 1.
2. **Certainty:** Something must happen (total probability = 1).
3. **Additivity:** Probabilities of mutually exclusive events add up.

Three simple rules. Every probability calculation in every LLM obeys them.

Maximum Likelihood — Finding the Best Fit

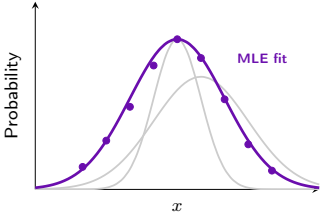
Ronald A. Fisher (1890–1962)

Origin: London, England

Introduced maximum likelihood estimation (concept 1912, formalized 1922). His 1922 paper "On the Mathematical Foundations of Theoretical Statistics" established MLE as a foundational method.

Recipe: Maximum Likelihood

Given the data I observed, which parameters make this data **most probable**?



Key Insight

LLM training is MLE at scale: find weights that maximize the probability of the training text. Fisher's method from 1922, applied to trillions of words.

Statistical Language Models — Shannon's Breakthrough (1948)

Claude Shannon (1916–2001)

Origin: Petoskey, Michigan, USA

Published *A Mathematical Theory of Communication* (1948), founding information theory. Modeled English as a stochastic process and built **n-gram language models** decades before AI.

Shannon (1948): predict next letter using previ-

Shannon's Approximations to English

0th order (random letters):
`XFOML RXKHRJFFJUJ`

1st order (letter frequencies):
`OCRO HLI RGWR NMIE`

2nd order (letter pairs):
`ON IE ANTSOUTINYS`

Word-level bigram:
`THE HEAD AND IN FRONTAL ATTACK`

More context ⇒ more readable!

Key Insight

Shannon was doing language modeling in

The Softmax Function — From Physics to Token Prediction

Ludwig Boltzmann (1844–1906)

Origin: Vienna, Austria

His Boltzmann distribution (1870s) describes the probability of a physical system being in a state based on its energy: $P \propto e^{-E/kT}$.

Lineage: Boltzmann (1870s, physics) → Gibbs (1900s, statistical mechanics) → Luce (1959, psychology of choice) → **Softmax** (ML)

Recipe: Softmax

Logits: $[2.1,\ 0.8,\ 0.3,\ -1.5]$

↓

Exp: $[8.17,\ 2.23,\ 1.35,\ 0.22]$

↓

Normalize:
$[0.68,\ 0.19,\ 0.11,\ 0.02]$

Formal

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$\sum$ = "add up for all j"

Building the Transformer — Probability & Statistics

Convergence diagram: Sections 1–2 lit (Linear Algebra in blue, Calculus in green). Section 3 "Probability & Statistics" now lights up in orange/violet. Label: "Softmax = next-token prediction."

Section 3 Summary

Who / When	Contribution
Pascal / Fermat (1654)	Birth of probability
Bernoulli (1713)	Law of large numbers
Bayes (1763) / Laplace	Updating beliefs from evider
Kolmogorov (1933)	Rigorous axioms
Fisher (1922)	Maximum likelihood estimat
Shannon (1948)	Language as a statistical pro
Boltzmann (1870s)	→ Luce (1959) → Softmax

Key Insight

Every time an LLM generates a word, it

Information Theory
Measuring Meaning

1865–1948

Clausius · Boltzmann · Shannon · Kullback · Zipf

Entropy — Invented Three Times

Clausius (1865)

Steam engine diagram

Entropy as "energy unavailable for work." Named from Greek *entropia* (transformation).

Boltzmann (1870s)

Gas molecules in a box; Boltzmann's tombstone with $S = k \log W$

Entropy as "number of arrangements."

$$S = k \log W$$

Shannon (1948)

Fair coin vs loaded coin; entropy formula diagram

Entropy as "average surprise."

$$H = -\sum_i p_i \log p_i$$

$\sum$ = "add up for every outcome i"

The same formula. Three completely different domains.

Cross-Entropy — THE Loss Function of LLMs

Recipe: Cross-Entropy Loss

1. The true answer is one specific word.
2. Look at the probability your model gave that word.
3. Take the logarithm.
4. Negate it.

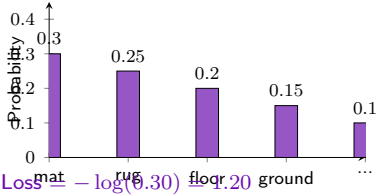
High confidence in right answer ⇒ low loss.
Low confidence in right answer ⇒ high loss.

Formal

$$\text{Loss} = -\log\big(p_{\text{model}}(\text{correct token})\big)$$

Full form uses $\sum$ over all tokens, but since only one

"The cat sat on the _____"



Loss $= -\log(0.30) = 1.20$

If model improves to 80% for "mat": loss $= -\log(0.80) = 0.22$

Key Insight

Training an LLM = minimizing cross-entropy = making the model more confident about the RIGHT next word.

Surprisal and Perplexity — How We Evaluate LLMs

Surprisal

Surprisal $= -\log(p(\text{word}))$

High probability $\Rightarrow$ low surprisal
Low probability $\Rightarrow$ high surprisal

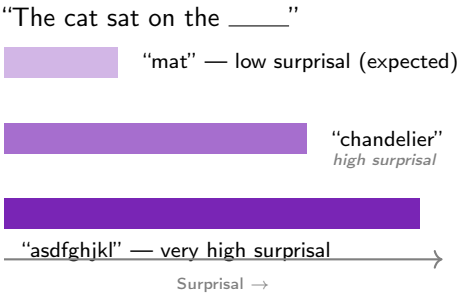
Perplexity

Average surprisal, exponentiated.

Lower perplexity = better model.

"GPT-4 has lower perplexity than GPT-3" means it's less surprised by real text.

Key Insight



KL Divergence — Measuring the Gap Between Distributions

Solomon Kullback (1907–1994) & **Richard Leibler** (1914–2003), USA.

Published KL divergence in 1951.

Recipe: KL Divergence

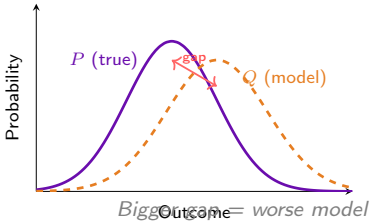
For each possible outcome, ask:
"How much more surprising is this under model Q compared to truth P?"

Average over all outcomes.

If Q matches P perfectly $\Rightarrow$ KL $= 0$.

Key Insight

In RLHF (the technique that makes ChatGPT helpful), KL divergence pre-



Formal

$$D_{\mathsf{KL}}(P\|Q) = \sum_i P(i)\,\log\frac{P(i)}{Q(i)}$$

Tokenization — Carving Language into Pieces

George Kingsley Zipf (1902–1950, USA).
Zipf's law (1935/1949): word frequency is inversely proportional to its rank. "The" is #1, "of" is #2, ...

Rico Sennrich, **Barry Haddow**, **Alexandra Birch** (Edinburgh, 2016).
Adapted Byte Pair Encoding (BPE) for neural machine translation. BPE is the tokenization method used by GPT and most modern LLMs.

Key Insight

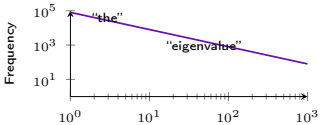
The AI doesn't see words — it sees tokens. BPE gives short tokens to common patterns (Zipf's law!) and splits

BPE Tokenization Example

Input:
`The mathematician calculated the eigenvalue`

Tokens:
`The` `mathematic` `ian` `calculated` `the` `eigen` `value`

Common words → single token
Rare words → split into pieces



Building the Transformer — Information Theory

~~Convergence diagram:~~ *Sections 1–3 lit (Linear Algebra, Calculus, Probability). Section 4 "Information Theory" now lights up in red. Label: "Cross-entropy = the loss function."*

Section 4 Summary

Who / When	Contribution
Clausius (1865)	Thermodynamic entropy
Boltzmann (1870s)	Statistical entropy, $S = k \log$
Shannon (1948)	Information entropy, the "bit
Kullback–Leibler (1951)	Divergence between distribu
Zipf (1935/1949)	Word frequency laws
Sennrich et al. (2016)	BPE tokenization

Key Insight

LLMs are trained by minimizing cross-entropy loss. Shannon's 1948 formula is

Logic & Computation
The Substrate

1854–1970s

Boole · Frege · Gödel · Turing · von Neumann

"Logicians dreamed of mechanical thought.
They proved it was impossible.
Then neural networks did it anyway."

Boolean Algebra — The Language of Computers (1854)

George Boole (1815–1864)

Origin: Lincoln, England; worked at Queen's College Cork, Ireland

Published *An Investigation of the Laws of Thought* (1854). Self-taught mathematician who showed that **logical reasoning** could be expressed as **algebra**: variables take values 0 (false) or 1 (true).

AND

A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

OR

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

NOT

A	Out
0	1
1	0

Every GPU in every AI data center is made of *billions* of these gates.

Key Insight

The Dream and Its Limits — From Frege to Turing

After Boole, logicians dreamed of reducing ALL reasoning to mechanical symbol manipulation. They almost succeeded — and then proved it was **impossible**.

Frege (1879)
First formal logical system.
A precursor to programming languages.

→

Russell & Whitehead
Principia (1910–13)
362 pages to prove $1+1=2$.

→

Gödel (1931)
Incompleteness: true statements exist that *cannot* be proved.

→

Turing (1936)
Halting problem: no algorithm can decide if any program will stop.

Key Insight

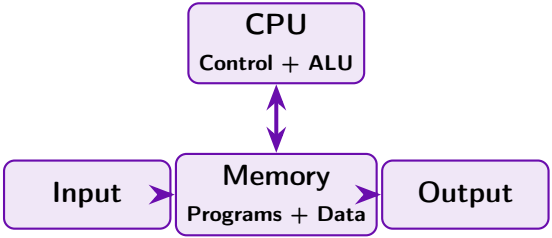
The irony: neural networks, built from Boole's logic gates, learned to translate, reason, and create — not by following rules, but by learning patterns from data. The machines that logic built went beyond what logic can do.

Von Neumann Architecture — The Hardware Blueprint (1945)

John von Neumann (1903–1957)

Born: Neumann János Lajos
Origin: Budapest, Hungary; worked at Princeton, USA

Published *First Draft of a Report on the EDVAC* (1945), describing the stored-program architecture: instructions and data share the same memory.



This basic design has powered nearly every computer for 80 years.

[EXTENDED] — can be cut; summarize

Floating Point — How Computers Handle Real Numbers

Konrad Zuse (1910–1995, Berlin, Germany). Built the Z3 (1941), the first programmable computer using floating-point arithmetic.

How 3.14159 is stored

sign	exponent (8 bits)	mantissa (23 bits)

IEEE 754 standard (1985)

Key Insight

Every weight in an LLM is a floating-point number. Large models have hundreds of billions of them.

Precision vs. Speed Trade-off

Format	Bits	Use
FP32	32	Traditional training
FP16 / BF16	16	Modern LLM training
INT8	8	Fast inference
INT4	4	Aggressive compression

Fewer bits = less precise but faster and smaller. Modern LLMs accept small rounding errors in exchange for fitting bigger models.

From Zuse's Z3 (1941) to today's GPU clusters: the same fundamental idea, vastly more scale.

GPU Computing — Why Matrix Multiplication Won

NVIDIA released CUDA (2007): general-purpose computing on GPUs.

Alex Krizhevsky, **Ilya Sutskever**, **Geoffrey Hinton** — AlexNet (2012): GPUs could train deep neural networks dramatically faster, launching the deep learning era.

Key Insight

Without GPUs, deep learning would still be a curiosity. Video game hardware accidentally enabled AI.

"AlexNet showed GPUs could train networks 10–100× faster than CPUs. That single

CPU vs GPU comparison:
CPU: 4–16 powerful cores
GPU: 5,000–16,000 simple cores
Neural networks = parallel matrix math = perfect for GPU

Neural network training: millions of *independent* matrix multiplications $\Rightarrow$ perfect for GPU.

Building the Transformer — Logic & Computation

Convergence diagram: Sections 1–4 lit (Linear Algebra, Calculus, Probability, Information Theory). Section 5 "Logic & Computation" now lights up in gray/steel. Label: "GPUs = the hardware."

Key Insight

Boolean logic built the hardware. Turing defined computation. GPUs made deep learning possible. And neural networks surpassed what for-

Section 5 Summary

Who / When	Contribution
Boole (1854)	Boolean algebra
Frege (1879)	Formal logic
Russell/Whitehead (1910–13)	*Principia Mathema*
Gödel (1931)	Incompleteness the
Turing (1936)	Computability, Turi
Von Neumann (1945)	Stored-program arc
Zuse (1941)	Floating-point com
CUDA (2007) + AlexNet (2012)	GPU revolution

Session 2 Complete

"We have the math and the hardware. But can we actually build a brain?"

Function Approximation

Why Neural Networks Work at All

1807–1991

Fourier · Weierstrass · Cybenko · Hornik

A neural network can approximate *any* function.
Here is why that is remarkable.

Joseph Fourier — Any Function as a Sum of Waves

Jean-Baptiste Joseph Fourier (1768–1830)

Origin: Auxerre, France

Key work: *Théorie analytique de la chaleur* (1822, based on work from 1807). To solve the heat equation, Fourier claimed that any function could be decomposed into sines and cosines.

Lagrange objected. The claim was initially controversial — and proved enormously fertile.

Fourier approximation of a square wave: 1 sine (rough), 3 sines (better), 10 sines (close), 100 sines (nearly perfect). Caption: "Any function = a sum of simple waves. Fourier, 1807."

Karl Weierstrass — Polynomials Can Do It Too [EXTENDED]

Karl Weierstrass (1815–1897)

Origin: Ostenfelde, Westphalia, Germany

Key result (1885): For any continuous function f on a closed interval and any desired accuracy $\varepsilon > 0$, there exists a polynomial P that stays within ε of f everywhere on that interval.

Key Insight

Polynomials are **universal approximators** for continuous functions. A century later,

A wiggly continuous function with polynomial approximations of increasing degree overlaid. The polynomials get closer and closer. Caption: "Any continuous function can be approximated as closely as you want by a polynomial."

The Universal Approximation Theorem

George Cybenko (b. c. 1952)

Origin: Canada/USA

Key result (1989): A feedforward network with a single hidden layer of sigmoidal neurons can approximate *any* continuous function to any desired accuracy.

"Approximation by Superpositions of a Sigmoidal Function"

Kurt Hornik (b. c. 1963)

Origin: Austria

A simple neural network with one hidden layer. As hidden neurons increase (5, 20, 100, 1000), the function it represents gets more complex and accurate. Caption: "More neurons = better approximation. Cybenko, 1989."

A neural network **can** learn any continuous function, given enough neurons. This is not an analogy. It is a *mathematical theorem*.

Convergence Diagram — Function Approximation Lights Up

Convergence diagram: "Function Approximation" node now lit in purple. 6 of 10 nodes active. Label: "Universal approximation = why it works."

Section 6 Summary

Fourier (1807/1822)	Any function as waves
Weierstrass (1885)	Polynomial approximation
Cybenko (1989)	Neural net universality
Hornik (1991)	Generalized activation

→ Connection to the Transformer

The universal approximation theorem guarantees that neural networks can represent any continuous function. The LLM's ability to model language is a specific case.

The Neuron & Depth

From Biology to Deep Networks

1943–2016

McCulloch & Pitts · Rosenblatt · Minsky · Hinton · He

A real neuron fires or does not. A mathematical neuron does the same thing, with an equation. But stacking them deep required more invention.

McCulloch & Pitts — The First Mathematical Brain Cell

Warren McCulloch (1898–1969)

Origin: Orange, New Jersey, USA

Neurophysiologist who sought to understand the brain in logical terms.

Walter Pitts (1923–1969)

Origin: Detroit, Michigan, USA

Self-taught logician. A troubled prodigy who ran away from home and ended up working with McCulloch.

Together they published *"A Logical Calcu-*

The McCulloch-Pitts neuron: inputs x_1, x_2, x_3 with weights w_1, w_2, w_3. Sum $= w_1x_1 + w_2x_2 + w_3x_3$. If Sum $>$ threshold, output $= 1$, else output $= 0$. Biological neuron shown alongside for comparison.

Frank Rosenblatt — The Perceptron: A Machine That Learns

Frank Rosenblatt (1928–1971)

Origin: New Rochelle, New York, USA

Built the **Mark I Perceptron** (1957–1958) at the Cornell Aeronautical Laboratory. A physical machine (not a simulation) that could learn to classify images.

The *New York Times* reported it as a machine that could "walk, talk, see, write, reproduce itself, and be conscious of its existence."

Photo of the Mark I Perceptron hardware: a large machine with photocells. Below: the perceptron learning algorithm — present an example, compare output to desired output, adjust weights.

Key Insight

The XOR Problem — The Crash That Froze AI

Minsky (1927–2016) & Papert (1928–2016)

Marvin Minsky: New York City, USA
Seymour Papert: Pretoria, South Africa / USA

Published *Perceptrons* (1969): a rigorous mathematical analysis proving that a single-layer perceptron cannot learn XOR.

The math was correct. But the field interpreted it as: neural networks are a dead end.

Top: XOR truth table and scatter plot showing XOR is not linearly separable. "A single line cannot separate the 1s from the 0s. A perceptron can only draw a single line." Bottom: Timeline of the AI Winter, 1969–1986.

Multi-Layer Networks — The Comeback

1986: The Revival

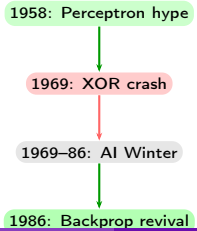
Rumelhart, Hinton & Williams (Slide 22) demonstrated that backpropagation could train multi-layer networks.

Multi-layer perceptrons CAN learn XOR — and much more. A hidden layer creates a *non-linear* decision boundary.

A multi-layer network solving XOR: input layer → hidden layer → output layer. The hidden layer creates a nonlinear boundary. Caption: "Two layers CAN separate XOR."

Key Insight

It took **17 years** for the field to recover from *Perceptrons*. The AI winter was caused by



Activation Functions — The Spice That Makes it Work

Key Insight

Without a nonlinear activation function, a multi-layer network collapses to a single layer. Activation functions give neural networks their power.

Sigmoid — $\frac{1}{1+e^{-x}}$

- Smooth S-curve, used 1980s–2000s
- Related to the logistic function of **Pierre François Verhulst** (1804–1849, Brussels), who modelled population growth (1838)

Three activation function curves plotted side by side: Sigmoid (smooth S-curve, labelled "1980s–2000s"), ReLU (hockey stick, labelled "2010s"), GELU (smooth ReLU, labelled "2016–present, used in GPT"). Caption: "These simple curves make neural networks nonlinear — and therefore powerful."

Residual Connections — The Trick That Enabled Depth

Kaiming He et al. (b. c. 1984)

Origin: China / USA

Key paper (2015): *"Deep Residual Learning for Image Recognition"* (ResNet).

Instead of each layer computing a completely new output, it computes a change to the input:

$$\text{output} = \text{input} + \text{layer}(\text{input})$$

This simple idea allowed networks to grow from ∼20 layers to 152 layers — and beyond.

Side-by-side comparison: Left — standard deep network with signal fading layer by layer (fading colour). Right — ResNet with skip connections; an arrow skips over each block, carrying the original signal forward. Below: residual formula as diagram: input → [layer] → ADD input back → output.

Layer Normalization & Dropout — Stabilising Deep Networks

Ba/Kiros/Hinton – Layer Norm (2016)

Adjusts the outputs of each layer so they have a consistent mean and variance. Stabilises training and speeds convergence.

Srivastava/Hinton et al. – Dropout (2014)

Randomly deactivates neurons during training, forcing the network to learn robust features rather than memorising.

Top: Layer normalisation — a row of neuron activations before (wildly varying) and after normalisation (centred, consistent). Caption: "Layer Norm = rescale so nothing explodes." Bottom: Dropout — a network with some neurons crossed out. Caption: "Dropout = randomly disable neurons

In the transformer's "Add & Norm":
Add = He's residual connection (Slide 60)
Norm = Ba's layer normalisation

Convergence Diagram — The Neuron & Depth Lights Up

Convergence diagram: "The Neuron & Depth" node now lit in yellow/gold. 7 of 10 nodes active. Label: "Activation + normalisation + residuals = deep networks."

Section 7 Summary

McCulloch-Pitts (1943)	First math neuron
Rosenblatt (1958)	Perceptron learns
Minsky/Papert (1969)	XOR, AI winter
Rumelhart/Hinton/W. (1986)	Multi-layer revival
Verhulst (1838) → Sigmoid	Population → activation
Nair/Hinton (2010)	ReLU
Srivastava et al. (2014)	Dropout
He et al. (2015)	Residual connections
Ba et al. (2016)	Layer normalisation

→ Connection to the Transformer

The artificial neuron, with its activation function, is the basic unit. Residual connections, layer normalisation, and dropout allow transformers to be DEEP. There are billions of neurons in GPT, stacked 96+ layers.

Sequence Modeling

The Path to Language

1906–2014

Markov · Baum · Elman · Hochreiter & Schmidhuber · Bahdanau

Language is a sequence. Modeling sequences required 100 years of invention.

Andrey Markov — The Simplest Language Model

Andrey Andreyevich Markov (1856–1922)

Origin: Ryazan, Russia

Introduced **Markov chains** (1906) by analysing the alternation of vowels and consonants in Pushkin's poem *Eugene Onegin*.

He was *not* studying language — he was disproving a claim that the law of large numbers required independence. But his method became the foundation of all sequence modelling.

Top: State diagram — states are words ("the", "cat", "sat", "on"), arrows labelled with transition probabilities. Bottom: Text generated by a Markov chain vs. text from an LLM. Markov text is grammatical locally but incoherent globally.

Hidden Markov Models — Adding Depth

Leonard Baum et al. (1960s–1972)

Origin: USA

Developed the mathematical framework for **Hidden Markov Models** (HMMs) and the **Baum–Welch algorithm** for training them.

HMMs add a hidden state layer: the observed output depends on an unobserved internal state that must be inferred.

HMM diagram: hidden states on top (connected by transition arrows), observed outputs below (connected to hidden states by emission arrows). Caption: "The hidden states cannot be seen directly — they must be inferred from observations."

Key Insight

Recurrent Neural Networks — Memory in a Loop

Michael Jordan – Jordan Network (1986)

Origin: USA

An early form of recurrent neural network. *Yes, the field has its own Michael Jordan.*

Jeffrey Elman (1948–2018) — Elman Network (1990)

Origin: USA

The hidden state at time t is fed back as input at time $t+1$. This is the standard

An RNN unrolled across time steps: the same network processing word 1, then word 2, then word 3, with the hidden state carried forward like a "memory" arrow. Caption: "The hidden state is the RNN's memory. But it is a very leaky memory."

LSTM — Long Short-Term Memory [EXTENDED]

Hochreiter & Schmidhuber

Sepp Hochreiter: Austria
Jürgen Schmidhuber: Munich, Germany

Published *"Long Short-Term Memory"* (1997).

Hochreiter's diploma thesis (1991, supervised by Schmidhuber) identified the vanishing gradient problem. The 1997 LSTM paper provided the solution: explicit **memory cells** with **gates** that control what to remember and what to forget.

Simplified LSTM cell: input gate, forget gate, output gate, cell state. Caption: "The forget gate decides what to remember. The input gate decides what is new. The output gate decides what to reveal."

The Attention Mechanism — The Bridge to Transformers

Bahdanau/Cho/Bengio (2014)

Dzmitry Bahdanau: Belarus/Canada
Kyunghyun Cho: South Korea/Canada
Yoshua Bengio: France/Canada

Published *"Neural Machine Translation by Jointly Learning to Align and Translate"* (2014 arXiv; 2015 ICLR).

Key insight: Instead of compressing the entire input into a fixed-size vector, let the model look back at all previous positions and decide which ones are relevant.

Top: Attention diagram — translating "Le chat est sur le tapis." When generating "cat," the model attends strongly to "chat." When generating "mat," it attends to "tapis." Colour-coded attention weights shown as lines of varying thickness. Bottom: Comparison — RNN (compresses everything into one vector) vs. Attention (can look back at everything).

The Transformer

Where Everything Converges

2017

Vaswani · Shazeer · Parmar · Uszkoreit · Jones
Gomez · Kaiser · Polosukhin

Eight branches of mathematics meet in one architecture.

"Attention Is All You Need."

“Attention Is All You Need” — The Paper That Changed AI

In 2017, a team at Google published a paper proposing an architecture that used **only attention** — no recurrence, no convolution. It was simpler, faster, and better.

The Eight Authors (2017 NeurIPS)

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Łukasz Kaiser, Illia Polosukhin

Most in their 20s and 30s. Several have since founded major AI companies (Cohere, Character.AI).

Top: The first page of “Attention Is All You Need” with title highlighted. Bottom: Stylised portraits or photos of the eight authors. Caption: “Eight researchers. One paper. One architecture.”

Self-Attention — The Core Mechanism

Each token generates three vectors: a **Query** ("what am I looking for?"), a **Key** ("what do I contain?"), and a **Value** ("what information do I carry?").

Self-Attention Step by Step

1. Compute attention scores: **dot product** of each Query with each Key (Gibbs, Slide 10)
2. Scale scores: divide by $\sqrt{d}$ to keep numbers manageable (cosine-like scaling, Slide 11)
3. Normalise with **softmax**: scores become probabilities (Boltzmann, Slide 33)
4. Output = weighted sum of Values via **matrix multiplication** (Cayley, Slide 9)

Formal notation (small inset):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Concrete example: In "The cat sat on the mat because *it* was tired," the word "it" attends most strongly to "cat." Self-attention learns that "it" refers to "cat."

Multi-Head Attention & Feed-Forward Networks

Multi-Head Attention

Instead of one set of Q/K/V, the transformer uses multiple "heads" (8, 12, or more) in parallel. Each head learns different relationships: syntax, semantics, position, etc. Results are concatenated and projected back.

Feed-Forward Network (FFN)

Two matrix multiplications with a GELU activation in between. This is the universal approximation theorem (Cybenko, Slide 52) applied within each transformer layer — the FFN provides the raw computational power.

A single transformer block, annotated: Multi-Head Attention sub-block → Add & Norm (residual + layer norm, Slides 60–61) → Feed-Forward Network sub-block → Add & Norm. Every component labelled with its origin section.

Key Insight

The Full Transformer – Annotated with 2,000+ Years of Math

Annotated components (back-references to earlier slides):

- ***Input Embedding:*** *Words as vectors (Grassmann 1844, Mikolov 2013, Slide 14)*
- ***Positional Encoding:*** *Sine/cosine (Fourier 1807, Slide 50)*
- ***Multi-Head Self-Attention:*** *Dot product + softmax + matrix mult. (Gibbs 1880s Slide 10, Boltzmann 1870s Slide 33, Cayley 1858 Slide 9, Bahdanau 2014 Slide 68)*
- ***Add & Norm:*** *Residual + Layer Norm (He 2015 Slide 60, Ba 2016 Slide 61)*
- ***FFN:*** *Matrix mult. + GELU (Cybenko 1989 Slide 52, Hendrycks 2016 Slide 59)*
- ***Dropout:*** *(Srivastava/Hinton 2014 Slide 61)*
- ***Output Softmax:*** *Probability over vocabulary (Fisher 1922)*
- ***Hardware:*** *GPU arrays (NVIDIA CUDA 2007, AlexNet 2012 Slide 47)*

Interactive Moment: "Count the Building Blocks"

Students: look at this diagram and try to identify which of the 10 sections each component comes from. How many can you find? *(3 min)*

Positional Encoding — Fourier Meets Transformers

Transformers have no built-in notion of word order (unlike RNNs). To fix this, they add **positional encodings** — sine and cosine functions at different frequencies. This is Fourier's idea, applied directly.

Recipe (CONCEPTUAL):

For each position in the sentence and each dimension in the vector, compute a sine or cosine wave with a specific frequency. Different dimensions use different frequencies — just like Fourier's decomposition. Each position gets a unique "fingerprint" of wave values.

Formal:

Heatmap of positional encodings: rows = positions (1st word, 2nd word, . . . , 100th word), columns = dimensions. Clear sinusoidal wave patterns visible. Caption: "Fourier's sine and cosine waves, used to tell the transformer which word is which."

Key Insight

Fourier decomposed heat into waves. Vaswani decomposed word position into waves. Same math, 210 years apart.

The Scaling Leap — From Transformer to LLM

The transformer architecture, scaled up with more parameters, more data, and more compute, produces Large Language Models with emergent abilities.

The GPT Series

GPT-1 (2018)	117M params	Pre-train + fine-tune
GPT-2 (2019)	1.5B params	Convincing text gen.
GPT-3 (2020)	175B params	Few-shot learning
GPT-4 (2023)	undisclosed	Multimodal
Claude (2023–26)	—	Constitutional AI

Jared Kaplan et al. (2020) — Scaling Laws

LLM performance scales as a smooth **power law** with com-

Top: Log-log plot of parameter count vs. year: GPT-1 (2018) → GPT-2 (2019) → GPT-3 (2020) → PaLM (2022) → GPT-4 (2023) → frontier models (2025–26). Exponential growth visible. Bottom: Timeline of key milestones: RLHF (2017), InstructGPT (2022), ChatGPT (Nov 2022), GPT-4 (Mar 2023), Claude 3 (2024), open-source explosion (Llama, Mistral).

Return to the Exploded Diagram — Now You Understand Every Piece

The SAME exploded transformer diagram from Slide 3, now with every label including the mathematician's name, date, AND slide number:

• Matrix multiplication (Cayley, 1858 — Slide 9) • Dot product (Gibbs, 1880s — Slide 10)

• Gradient descent (Cauchy, 1847 — Slide 21) • Softmax (Boltzmann, 1870s — Slide 33)

• Cross-entropy (Shannon, 1948 — Slide 37) • Backpropagation (Linnainmaa 1970, Rumelhart/Hinton/Williams 1986 — Slide 22)

• Activation functions (Verhulst 1838, Nair/Hinton 2010 — Slide 59)

• Residual connections (He et al. 2015 — Slide 60) • Layer norm (Ba et al. 2016 — Slide 61)

• Self-attention (Bahdanau 2014, Vaswani 2017 — Slides 68, 71)

• Positional encoding (Fourier 1807 — Slide 74) • GPU computing (NVIDIA CUDA 2007 — Slide 47)

Convergence Diagram — Final State

The completed convergence diagram. All 10 nodes lit. Central "TRANSFORMER" node glowing. Each node label includes its cumulative date range:

1. Linear Algebra (1809–2013) — blue
2. Calculus & Optimisation (c. 250 BCE–1986) — green
3. Probability & Statistics (1654–1959) — orange
4. Information Theory (1865–2016) — red
5. Logic & Computation (1854–2012) — grey
6. Function Approximation (1807–1991) — purple
7. The Neuron & Depth (1943–2016) — yellow
8. Sequence Modeling (1906–2014) — teal
9. The Transformer (2017) — multicolour
10. Scaling to LLMs (2018–2020) — gradient

Ten streams of human invention, flowing into one machine.

The Story Is Not Over

∞ 2,000+ Years of Mathematics. One Machine. Your Turn.

- Every component of an LLM was invented by someone, for a specific purpose, often centuries before AI existed.
- Grassmann imagined abstract vector spaces. Boltzmann studied gas molecules. Fourier solved the heat equation. Markov analysed poetry. Archimedes approximated circles. **None of them imagined AI.**
- The math you learn in school — algebra, calculus, probability — is the *same math* running inside the AI you talk to every day.
- The next building block has not been invented yet. It might come from pure mathematics, from physics, from linguistics, from neuroscience — **or from someone in this room.**